Data Assessment Prior Processing

Motivation for Data Analysis and Cleanup

- Once the data are available from a study there are still some steps required to get them into proper shape for analysis.
- This is particularly true as multivariate analyses are often carried out on **large data sets**
- Information contained in large datasets are processed by MVDA techniques in order to draw inferences
- If some data are missing or erroneous then conclusions may be compromised

How to Do Data Cleanup?

- Spot missing values using visual means (plots) and check for absurd entries
- Missing values should be made up/filled in with reasonable values
- Method used to fill missing data may depend on the trend shown by existing data
- In thermal monitoring, slowly changing T may suggest using the nearest valid value
- Data showing seasonality like weather may apply statistical methods such as moving average or median
- Data sets exhibit strong relationship with previous values e.g. stock prices. Use interpolation-based techniques to fill in missing data.

https://in.mathworks.com/discovery/data-cleaning.html

```
Data = [

 1    2     NaN

 5   NaN    7    ];

out = ismissing(Data);

cleanData = rmmissing(Data);
```

```
>> MissingData_Eg

out =

  2x3 logical array

   0    0    1
   0    1    0
```

```
>> cleanData

cleanData =

  0x3 empty double matrix
```

```
Data = [
 1   2   NaN
 5  NaN   7   ];
out = ismissing(Data);
cleanData = rmmissing(Data);
```

```
>> MissingData_Eg

out =

  2x3 logical array

   0   0   1
   0   1   0
```

```
>> cleanData

cleanData =

  0x3 empty double matrix
```

Depending on your input data structure, rmmissing executes different clean-up rules automatically

Vectors: It deletes any individual elements that are missing, shortening the vector.

Matrices & Tables: It deletes any **entire row** that contains at least one missing value

Python

```
import numpy as np

data = np.array([ [1, 2, np.nan], [5,

np.nan, 7] ])

out = np.isnan(data)

cleanData =

data[~np.isnan(data).any(axis=1)]
```

```
out
```

```
array([[0, 0, 1],
       [0, 1, 0]])
```

```
cleanData
```

```
array([], shape=(0, 3), dtype=float64)
```

Depending on your input data structure, rmmissing executes different clean-up rules automatically

Vectors: It deletes any individual elements that are missing, shortening the vector.

Matrices & Tables: It deletes any **entire row** that contains at least one missing value

Data Cleaning

- Dirty data may mislead AI models, leading to bias and error in their predictions
- Data cleaning removes noise, inconsistent data, missing values, and outliers
- After cleaning input data, AI algorithms are able to learn valid patterns and predict accurately at new points

https://in.mathworks.com/discovery/data-cleaning.html

Missing Data: Vexing Issue

- If you have a set of observations/features involving several columns (several features) and several observations (large data set) and to your chagrin find that some cells (i^{th} observation of j^{th} feature) to be missing then what to do?
- Data may be missing or invalid due to faulty sensors not throwing any output, nonsensical data, wrongly entered data (e.g. weight as -5 kg), etc.
- Missing data may vex data analysts as they may cause incorrect simulations and parameter estimation, inaccurate correlations and models, leading eventually to wrong decision making

Perils of Missing Data

- For e.g., missing/erroneous data may cause a manager make a wrong decision regarding hiring, loan sanctioning, investing in a new project, wrong choice of material for an engineering application, etc.
- Data cleaning and filling carefully missing data, are essential for ensuring integrity of data and reliable data processing outcomes

Missing at Random (MAR)

- Occurs when the probability of data becoming missing is related to **other observed information**, but not the missing data itself.
- In a solar installation involving solar irradiance, voltage, frequency etc., may not show grid voltage and frequency if due to night, rain, clouds, solar irradiance is insufficient to power up the system (https://in.mathworks.com/discovery/missing-data.html)
- In a social study where men, unlike women are less likely to report their income, but the missing of the data is accounted for by the observed "Gender" column, not the income value itself

Blame other Features/Variables

Missing at Random (MAR): More Examples

- In a questionnaire, contract workers miss entering their salaries than regular workers
- So the missingness is not attributed to the value of the income, but to the category of the employee
- Young adults are more likely to skip questions about their annual income. The probability of the data being missing is fully explained by the observed variable "age," not the actual amount of income they earn.
- In an entrance exam, the missing answer to a question may be explained by rural or urban student and not to the difficulty of the question itself
- Follow up to a medical treatment may be missed by men than women and may not be attributed to the seriousness of the disease by the patient but by patient gender

Blame other Features/Variables

Missing Completely at Random (MCAR)

The underlying reason for missing values is completely unrelated to any other variable in the data set.

Examples are given below.

- High channel noise and faulty sensors cause missing information segments in weather telemetry
- **Malfunctioning Sensor:** In a climate study, a temperature sensor occasionally fails to record data at random times, making the missing data independent of the actual temperature at that time
- A blood sample is accidentally spilled or destroyed in the lab during testing, causing that specific patient's results to be missing – irrespective of when the sample was taken or what medical history the patient had when giving the test

Missing Completely at Random (MCAR): More Examples

- MCAR data occurs when the absence of data is entirely unrelated to any observed or unobserved variables, meaning the missing values are just a random subset of the total data.
- Common examples include equipment failures, accidental, unsystematic errors, malfunctioning sensor/equipment
- In a weather study, a temperature sensor stops working at random times, failing to record data regardless of whether it is hot or cold.
- Survey Skip Mistakes: A survey respondent accidentally skips page 3 of a questionnaire simply because the pages stuck together, not because they disliked the questions

Blame not other features/variable for the missing data

Missing not at random (MNAR):

- The underlying cause of missing data is related to the variable itself.
- Probability of data missing is directly related to the unobserved value itself, creating a systematic, non-random bias.
- E.g.: if a sensor relaying temperature information has reached its measurement limits, it would result in missing values in the form of its saturated thresholds
- A sensor fails more often when the true temperature is very high
- Income Survey: Individuals with extremely high or low incomes are less likely to report them, causing the dataset to miss values at the extremes.

Value itself is responsible for it going abegging

Handling of Missing Data

- Identifying missing data is straightforward
- However, what is its appropriate replacement value?
- Plot the data to identify gaps
- Flag irregularities
- Instead of arbitrarily filling numbers in missing locations, do it intelligently depending on the context based on suitable procedures so that data's reliability does not come under the cloud

https://in.mathworks.com/discovery/data-cleaning.html

https://in.mathworks.com/help/ident/ug/handling-missing-data-and-outliers.html

Matlab Suggestions for Simple Situations

- For missing completely at random data (MCAR), simple imputation methods such as replacing missing values with the mean or median work well.
- Use the fillmissing function in MATLAB with 'constant' for quick solutions:

```
data_filled_mean = fillmissing(Data, 'constant', mean(data, 'omitnan'));
```

```
Data=[1 2 3 NaN 5 6 7];

data_filled_mean =     1     2     3     4     5     6     7
```

```
A = [1 3 NaN 4 NaN NaN 5];

F = fillmissing(A,"previous")

F =     1     3     3     4     4     4     5
```

Matlab Suggestions for Simple Situations

```
>> Data2

Data2 =

     1   NaN     2
     4     5   NaN
   NaN     2     3
```

```
data2_filled_mean =

    1.0000    3.5000    2.0000
    4.0000    5.0000    2.5000
    2.5000    2.0000    3.0000
```

```
Data2 = [ 1 NaN 2; 4 5 NaN; NaN 2 3];

data2_filled_mean = fillmissing(Data2, 'constant', mean(Data2, 'omitnan'));
```

Matlab Suggestions for Simple Situations

- For larger data sets, machine learning–based methods such as fitrensemble may predict missing values based on other variables.
- How should I handle missing data in sensor or time series data?
- For sensor & time series data, interpolation methods such as linear/spline interpolation are ideal.
- You can use the fillmissing function in MATLAB with the 'linear' option to fill gaps effectively:

```
data_filled_interp = fillmissing(data, 'linear');
```

This will fill the NaN values in data by linearly interpolating the missing values based on the surrounding data points.

```
clc; clear all
Data=[1 2 3 NaN 5 6 7];
data_filled_mean = fillmissing(Data, 'constant',
mean(Data, 'omitnan'));
Data2 = [ 1 NaN 2; 4 5 NaN; NaN 2 3];
data2_filled_mean = fillmissing(Data2, 'constant',
mean(Data2, 'omitnan'));
data_filled_interp = fillmissing(Data, 'linear');
data2_filled_interp = fillmissing(Data, 'linear');
```

Data=[1 2 3 **NaN** 5 6 7];

data_filled_mean = 1 2 3 **4** 5 6 7

>> Data2

Data2 =

1	**NaN**	2
4	5	**NaN**
NaN	2	3

data2_filled_mean =

1.0000	**3.5000**	2.0000
4.0000	5.0000	**2.5000**
2.5000	2.0000	3.0000

data_filled_interp = fillmissing(Data, 'linear');

data2_filled_interp = fillmissing(Data, 'linear');

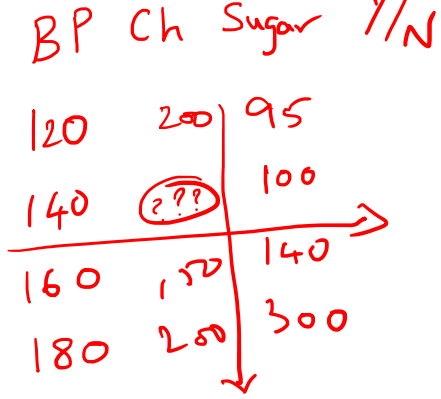
data_filled_interp =

1 2 3 **4** 5 6 7

>> data2_filled_interp

data2_filled_interp =

1.0000	**8.0000**	2.0000
4.0000	5.0000	**2.5000**
7.0000	2.0000	3.0000



```python
import numpy as np
import pandas as pd

print("--- 1D Data Results ---")
data = pd.Series([1, 2, 3, np.nan, 5, 6, 7])

# Mean Fill
data_filled_mean = data.fillna(data.mean())
print("Mean Fill:\n", data_filled_mean.values)

# Linear Interpolation
data_filled_interp = data.interpolate(method='linear')
print("\nLinear Interp:\n", data_filled_interp.values)

print("\n--- 2D Data Results ---")
data2 = pd.DataFrame([
    [1, np.nan, 2],
    [4, 5, np.nan],
    [np.nan, 2, 3]
])

# Mean Fill
data2_filled_mean = data2.fillna(data2.mean())
print("Mean Fill:\n", data2_filled_mean.values)

# Linear Interpolation (with Extrapolation for the edges)
data2_filled_interp = data2.interpolate(
    method='slinear',
    limit_direction='both',
    fill_value='extrapolate'
)
print("\nLinear Interp (with Extrapolation):\n", data2_filled_interp.values)
```

```
--- 1D Data Results ---
Mean Fill:
 [1. 2. 3. 4. 5. 6. 7.]

Linear Interp:
 [1. 2. 3. 4. 5. 6. 7.]

--- 2D Data Results ---
Mean Fill:
 [[1.  3.5 2. ]
 [4.  5.  2.5]
 [2.5 2.  3. ]]

Linear Interp (with Extrapolation):
 [[1.  8.  2. ]
 [4.  5.  2.5]
 [7.  2.  3. ]]
```

Matlab : Best Practices for Handling Missing Data

https://in.mathworks.com/discovery/missing-data.html

1. Identify Missing Data: Systematically detect missing data in your data set.
2. **Use visualization tools, statistical tests, and anomaly detection techniques to uncover patterns and assess the extent of missingness.**
3. Understanding where and why data is missing is the first step in choosing the right approach to address it.
4. Keep clear records of how missing data was handled in your analysis, including the type of missing data, the methods used to fill gaps, & any assumptions made
5. Transparency in documentation ensures reproducibility of your work which may be validated by others and strengthen the reliability of your results.

Matlab : Best Practices for Handling Missing Data

Engineers use statistical tests, …, assess the extent of missingness

Statistical Tests: Engineers check if missing data is (MCAR)

- **Two-Sample T-Tests:** Divide the dataset into two groups—

    1. with observed values and

    2. one with missing values for a specific feature — and run a t-test.

A statistically significant difference in means indicates the missingness may be non-random.

Matlab : Best Practices for Handling Outliers

Engineers use anomaly detection techniques to … assess the extent of missingness

- **Z-Score & Interquartile Range (IQR):** Measures how many standard deviations a data point is from the mean of the data set.
- Data points that exceed specific thresholds (e.g., |(Z > 3)| may be flagged as anomalies)
- **Moving Averages & Control Charts**: Smoothen expected fluctuations in time-series data so that sudden decreases or anomalies in data collection are easily identified
- **Data Visualization:** Tools like missing value heat maps and distribution charts visually highlight where gaps are concentrated in the dataset.

https://www.acceldata.io/blog/how-to-detect-statistical-anomalies-with-proven-methods

Python: Heat Maps For Missing Data

1. The Raw Data (What it looks like)

If you just print the dataframe, it looks like a tedious spreadsheet where `NaN` or `None` represents missing information:

User_ID	Age	Email_Verified	Monthly_Spe
101	28	True	$15
102	NaN	True	$0
103	34	NaN	$45
104	41	True	NaN
105	NaN	False	$12
106	22	True	$8
107	55	NaN	$80
108	NaN	True	NaN

https://colab.research.google.com/drive/12LsaMxhVGbtrAtAZsXiZO1v6c_6Cc37R#scrollTo=gWi9OqDiDx31

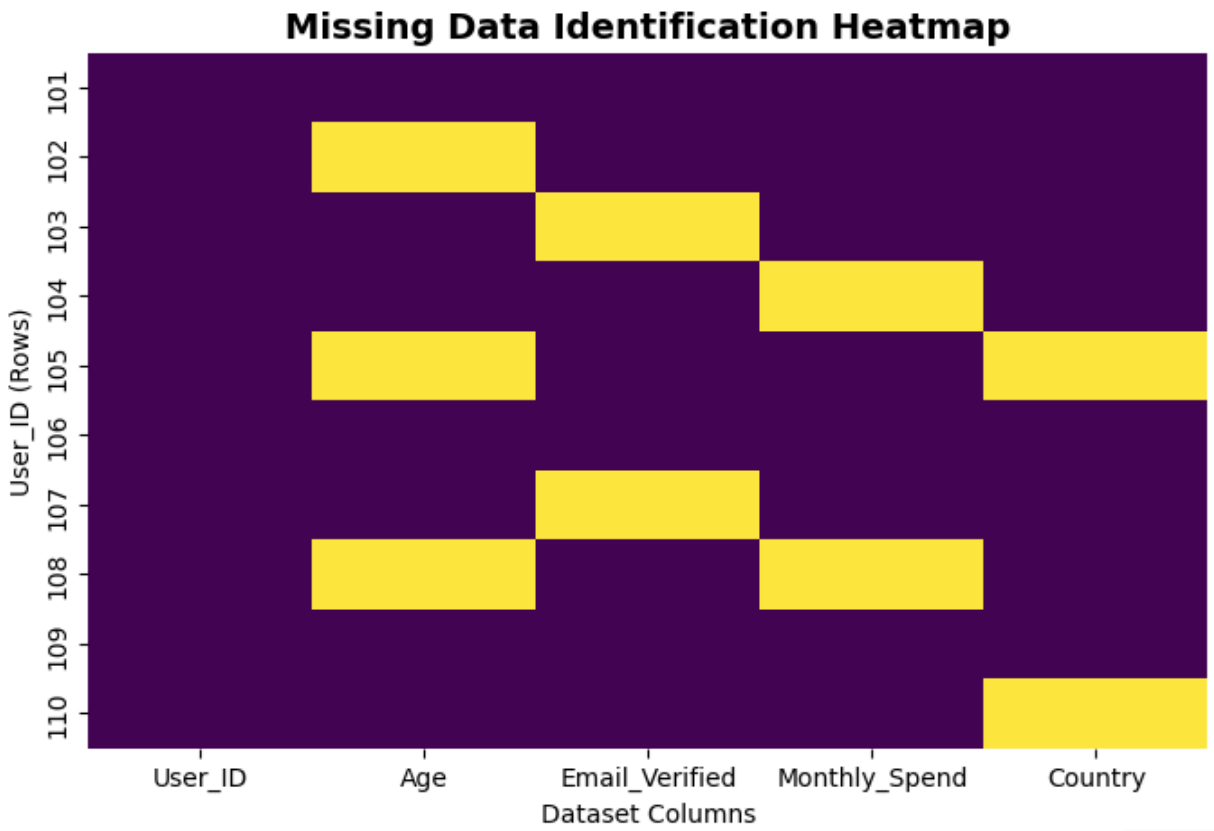
Python: Heat Maps For Missing Data

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# 1. Create the dummy dataset
data = {
    'User_ID': [101, 102, 103, 104, 105, 106, 107, 108, 109, 110],
    'Age': [28, np.nan, 34, 41, np.nan, 22, 55, np.nan, 29, 31],
    'Email_Verified': [True, True, np.nan, True, False, True, np.nan, True, False, True],
    'Monthly_Spend': [15, 0, 45, np.nan, 12, 8, 80, np.nan, 22, 19],
    'Country': ['USA', 'UK', 'Canada', 'Germany', np.nan, 'France', 'USA', 'Australia', 'India', np.nan]
}
df = pd.DataFrame(data)

# 2. Plot the colorful missing data heatmap
plt.figure(figsize=(8, 5))
# 'isnull()' creates the True/False matrix, 'cmap' sets the vibrant colors
sns.heatmap(df.isnull(), cbar=False, cmap='viridis', yticklabels=df['User_ID'])

plt.title('Missing Data Identification Heatmap', fontsize=14, fontweight='bold')
plt.xlabel('Dataset Columns')
plt.ylabel('User_ID (Rows)')
plt.show()
```



Python: Heat Maps For Missing Data

https://colab.research.google.com/drive/12LsaMxhVGbtrAtAZsXiZO1v6c_6Cc37R#scrollTo=gWi9OqDiDx31

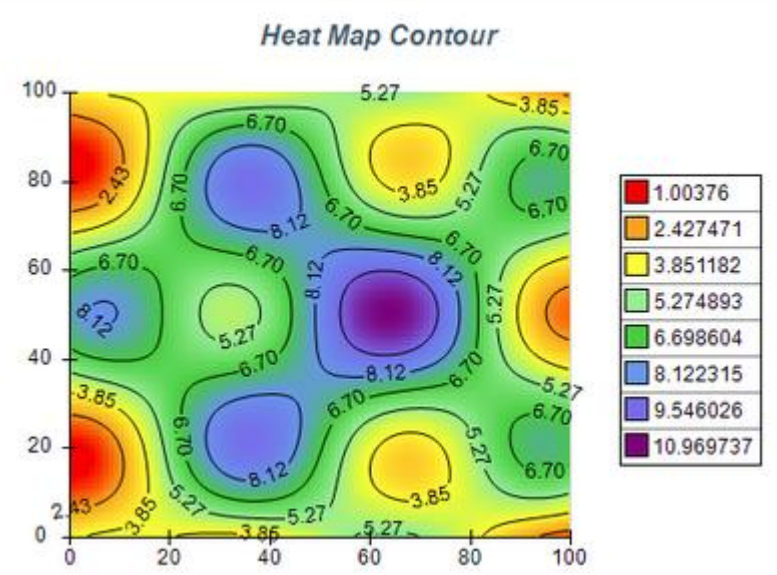
How to Read This Heatmap

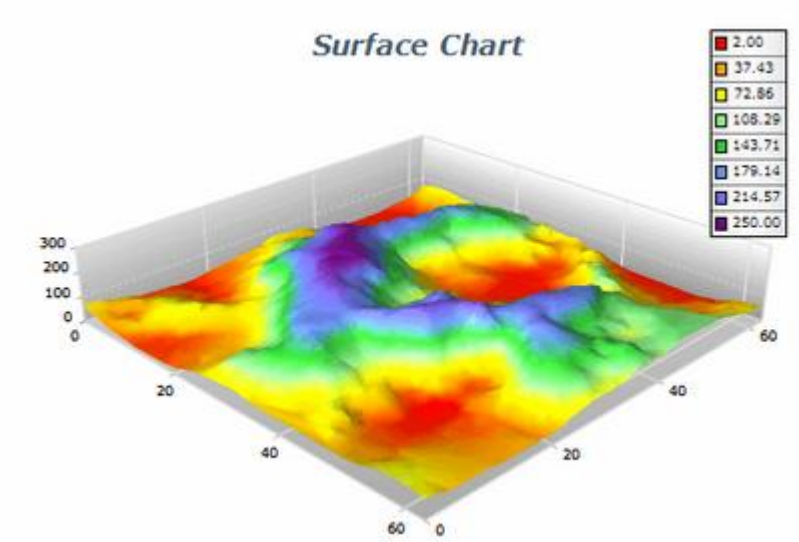
When you look at the resulting visualization, your brain instantly processes the patterns without reading a single number:

- **The Solid Blue Columns:** Look at `User_ID`. It is a solid block of deep navy. This tells you instantly that your primary key is 100% complete.
- **The "Swiss Cheese" Columns:** `Age` and `Monthly_Spend` look like they have bright neon horizontal slashes cutting through them. You can immediately see that about 30% of your user age data is missing.
- **The Warning Stripes:** Look at row 105 and 108. You'll see multiple neon blocks lining up horizontally. This tells you that specific *users* failed to fill out multiple parts of the form, rather than it being a systemic issue with just one column.

Matlab : Visualization Tools

Engineers use visualization tools, … assess the extent of missingness





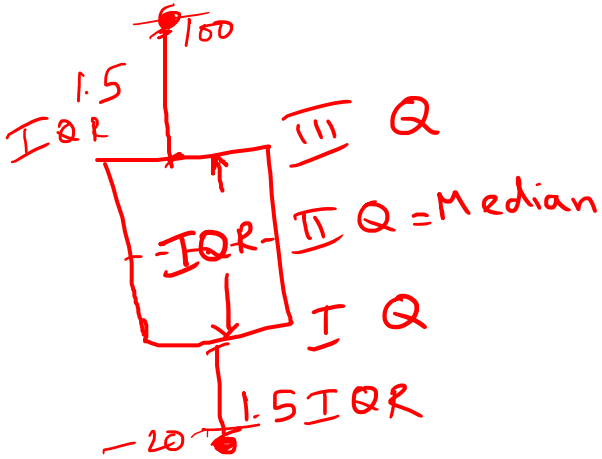
https://www.nevron.com/dotnet-heat-map-chart

Box and Whiskers Plots: Quartiles

a. **First quartile:** refers to the data point which has approximately 25% of the data below and 75% of the data above

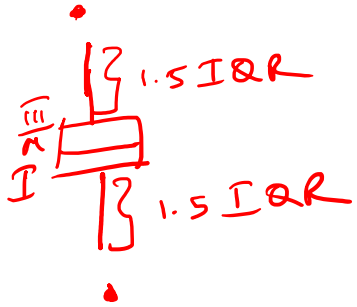
b. **Second quartile:** 50%, equal to median

c. **Third quartile:** 75% of the data below



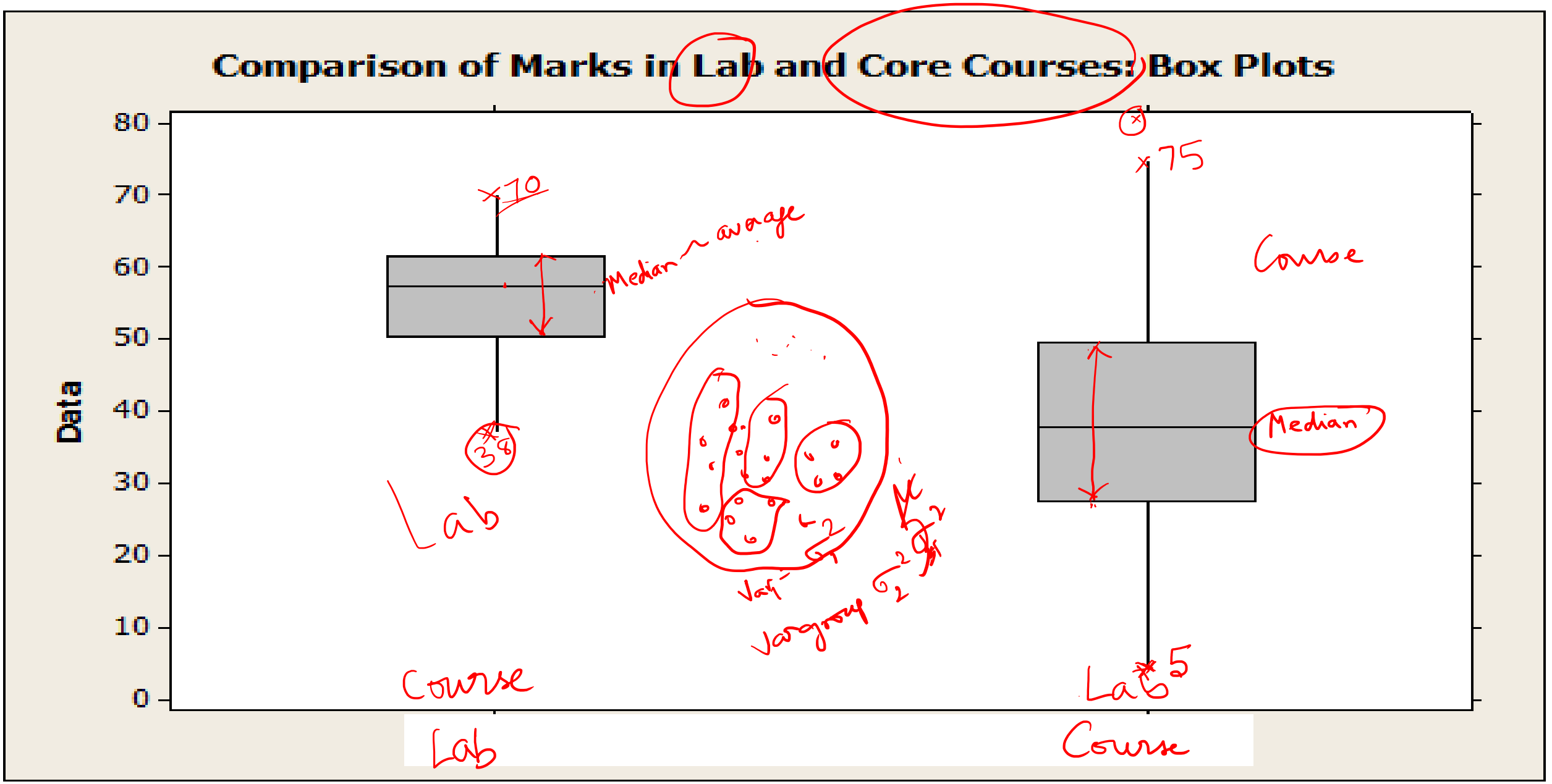
d. **Inter quartile range:** Difference between third and first quartiles

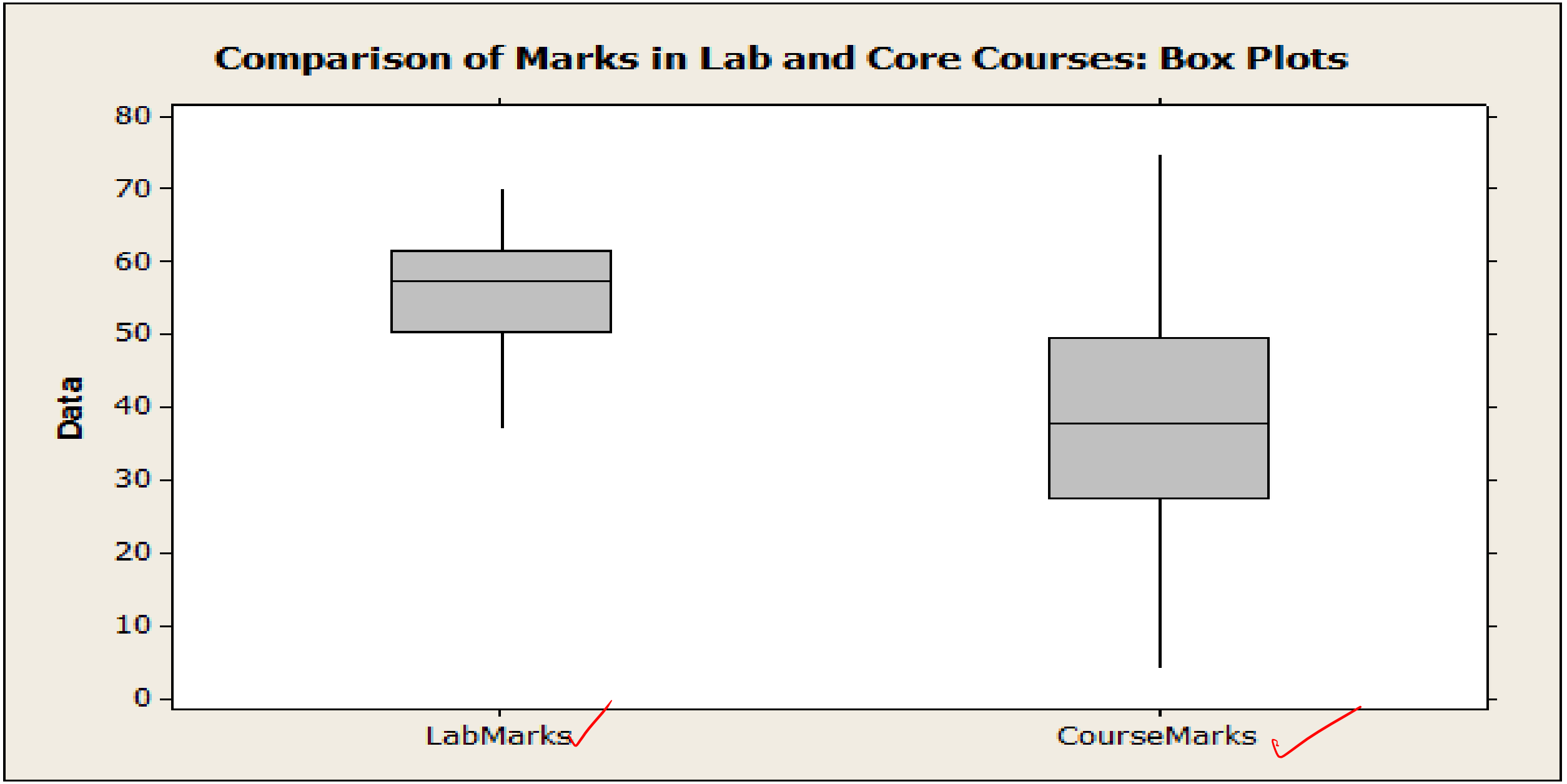
e. **Whiskers** extend to 1.5 times the inter quartile range on either side of the first and third quartile

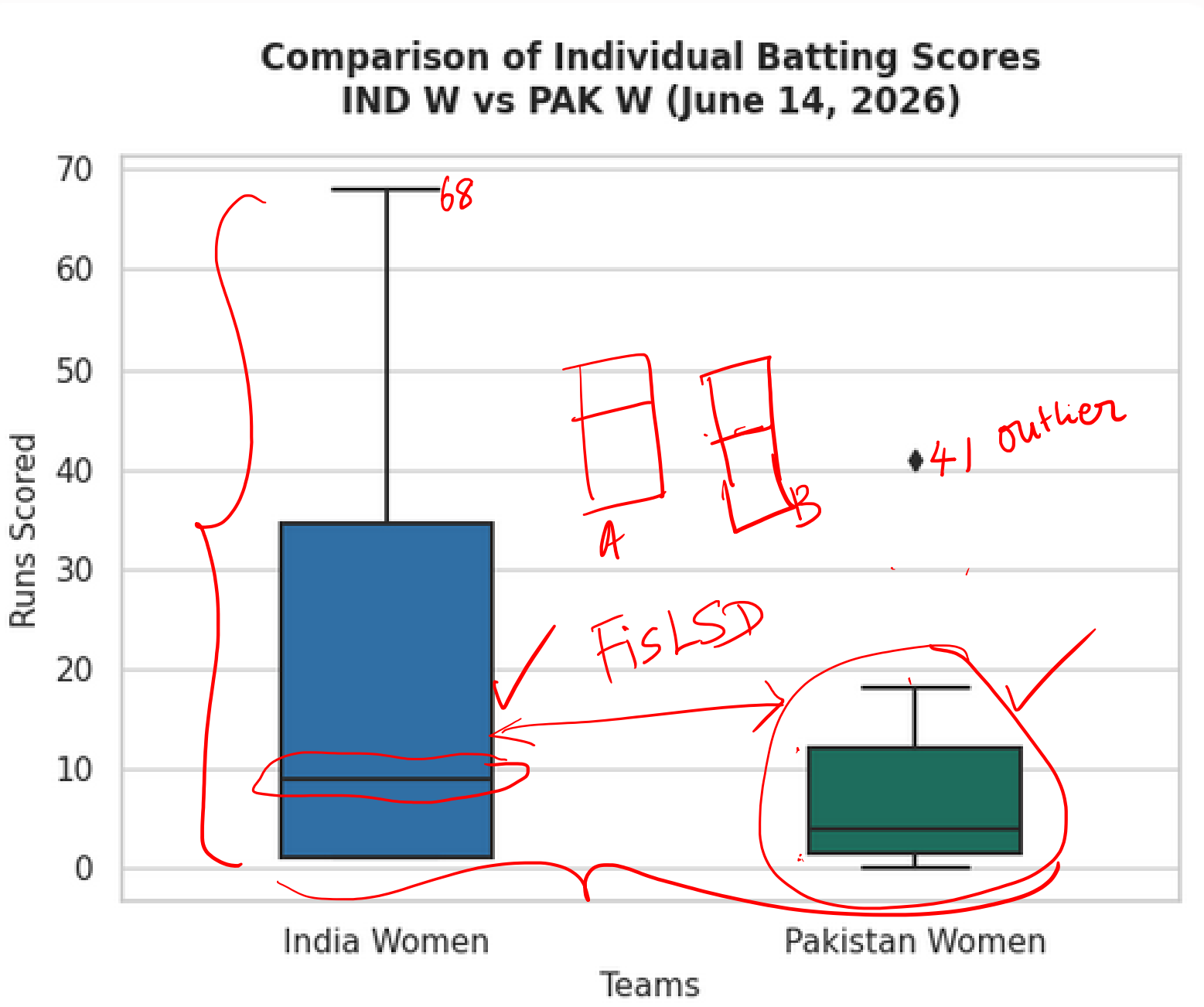


1. The zeroth quartile has the lowest value
2. The fourth quartile has the highest value
3. The whiskers originate from the first and third quartile and extend to 1.5 times the interquartile range.
4. The value below or above the respective whiskers are called outliers

10	0	10
20	1	47.5
30	3	122.5
40	4	160
50	2	85
60		
70		
80		
90		
100		
110		
120		
130		
140		
150		
160		


Comparison of Marks in Lab and Core Courses: Box Plots
Data
80
70
60
50
40
30
20
10
0
Course
Lab
Lab
Course
Median ~ average
Median
Course
Lab
Course
Var
Variance


Comparison of Marks in Lab and Core Courses: Box Plots
Data
80
70
60
50
40
30
20
10
0
LabMarks
CourseMarks



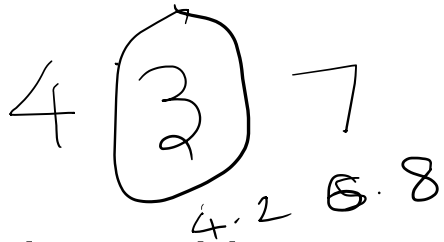
Total Variation = Variation within the Groups + Variation b/w Groups

Matlab : Best Practices for Handling Missing Data

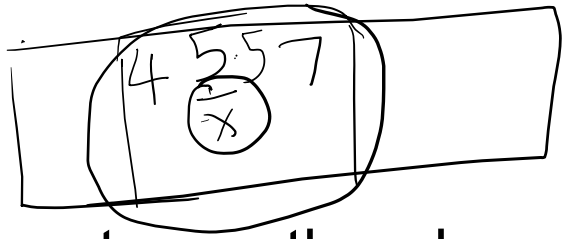
1. **Run Sensitivity Analyses:** Test how different methods for handling missing data affects your results.
2. **Compare outcomes** across different imputation or deletion techniques
3. **Assess stability** of the ensuing conclusions and identify the most reliable approach for the specific data set.
4. This minimizes unintended distortions in data analysis involving modeling, simulating, and optimizing as well as predicting
5. **Minimize Bias:** Improper management of missing data may introduce bias, leading to inaccurate conclusions and flawed engineering decisions

Imputation

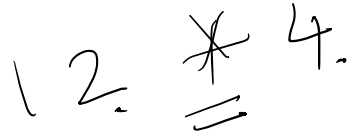
- Data analysis technique that replaces missing values in a dataset with estimated values.



- Imputation preserves all cases by replacing missing data with an estimated value based on other available information.



- Once all missing values have been imputed, the data set may then be analyzed using standard techniques for complete data.

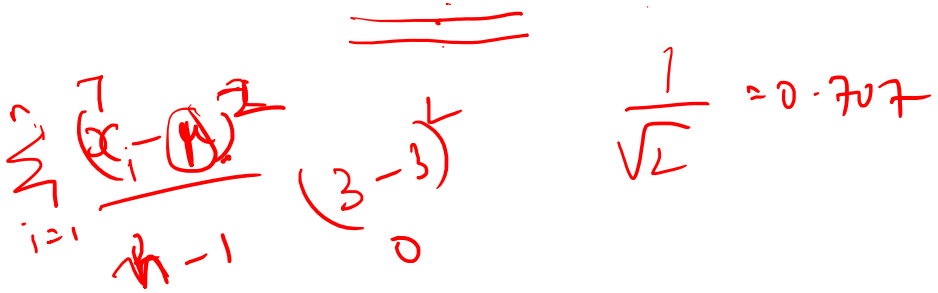


- It's a common way to handle missing data and is an important step in data preprocessing.

Mean, Median, or Mode Substitution

- Without any data, basic statistical central tendency measures such as the mean, median, or mode could provide the reasonable estimate to fill in the missing value.
- This approach results in artificially reducing the variance of the variable and could yield lower covariance and correlation estimates

A	A Mean	A Median
1	1	1
2	2	2
3	3	3
4	4	4
5	5	5
6	6	6
3.5	3	3.5
1.870829	1.718249	1.707825



Moving Mean Method: Example

```
% Sample data with missing values
A = [1, 2, NaN, 4, NaN, 6, 7, NaN, 9, 10]

% Fill missing values using a moving mean with a window length of 5
filledData = fillmissing(A, 'movmean', 5);

% Display the data after filling missing values
disp('Data after filling missing values using moving mean with window length of 5');
disp(filledData)
```

```
A = 1×10
     1     2   NaN     4   NaN     6     7   NaN     9    10

Data after filling missing values using moving mean with window length of 5
    1.0000    2.0000    2.3333    4.0000    5.6667    6.0000    7.0000    8.0000    9.0000   10.0000
```

Here computed missing values are not used in the calculations unlike in other cases

Moving Mean Method: Example - Python

```
import pandas as pd
import numpy as np

# Sample data with missing values
A = [1, 2, np.nan, 4, np.nan, 6, 7, np.nan, 9, 10]
s = pd.Series(A)

# Calculate the moving mean with a window length of 5.
# 'min_periods=1' ensures the mean is calculated even if there are NaNs in the window.
moving_mean = s.rolling(window=5, center=True, min_periods=1).mean()

# Fill only the missing values using the computed moving mean
filledData = s.fillna(moving_mean)

# Display the data after filling missing values
print("A = 1x10")
print(s.values)
print("\nData after filling missing values using moving mean with window length of 5")
formatted_output = [f"{val:.4f}" for val in filledData]
print("   ".join(formatted_output))
```

```
A = 1x10
[ 1.  2. nan  4. nan  6.  7. nan  9. 10.]

Data after filling missing values using moving mean with window length of 5
1.0000   2.0000   2.3333   4.0000   5.6667   6.0000   7.0000   8.0000   9.0000   10.0000
```

Moving Mean Method: Example

- Let us assume a window size of 5
- https://in.mathworks.com/discovery/data-cleaning.html

✅ **Step: Use a 5-point window around the missing value**

Take **2 values before and 2 values after** the missing point:

$$(4,\ 6,\ ?,\ 10,\ 12)$$

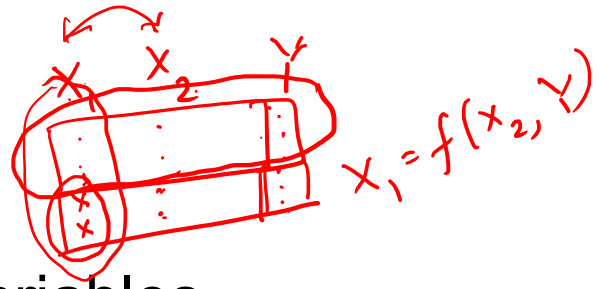
Now compute the mean **excluding the missing value**:

$$\text{Estimated value} = \frac{4 + 6 + 10 + 12}{4} = \frac{32}{4} = 8$$

📊 **Final filled data:**

$$[2,\ 4,\ 6,\ 8,\ 10,\ 12,\ 14]$$

Regression Imputation



- Also known as conditional mean imputation,
- This method models the missing value based on the other variables
- Suppose you are examining a regression model with multiple independent X variables.
- Further suppose that one independent variable, X_i, has missing values.
- You select all those observations with complete information, use X_i as the dependent variable (also called regress X_i), and use all the other Xs as independent variables to predict the missing values of X_i.

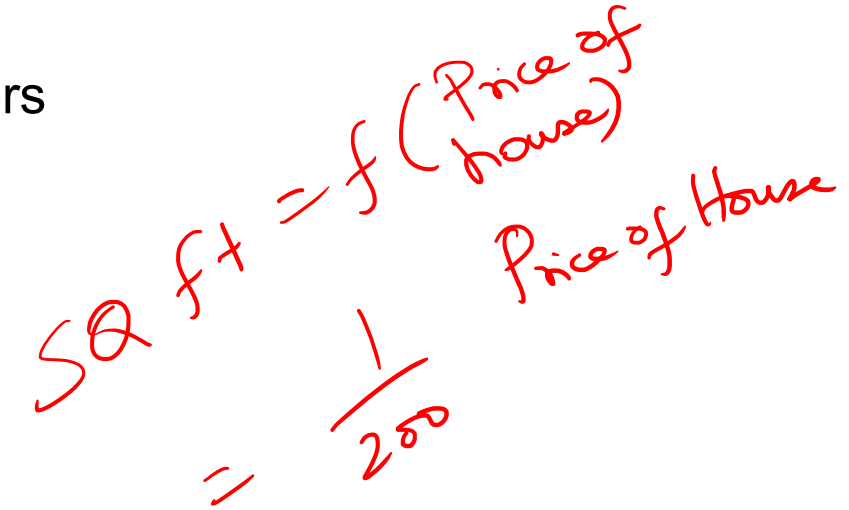
Regression Imputation

- This method reduces the variance of X_i and could produce higher covariance between the X variables and increase correlation values.
- In general, the major limitations of the regression imputation methods are that they lead to underestimates of the variance and may produce biased test statistics.

Regression Imputation

- A technique where missing values in a target variable are entered using predictions from a linear regression model
- To build the linear model, we use complete data of predictors

House	SqFt (x)	Price (y) - Missing
1	1000	200,000
2	1500	300,000
3	1800	**NaN** (Missing)
4	2000	400,000
5	2500	500,000
6	3000	**NaN** (Missing)
7	3500	700,000



Regression Imputation

🛠️ Step-by-Step Imputation

1. Train the Model

Using only the **complete cases** (Houses 1, 2, 4, 5, 7), we fit a linear regression:

$$\text{Price} = m \times \text{SqFt} + c$$

- **Result:** The model finds that $\text{Price} \approx 200 \times \text{SqFt}$.

2. Predict Missing Values

Using the model, we predict the prices for Houses 3 and 6:

- **House 3 (1800 SqFt):** $1800 \times 200 = \$360,000$
- **House 6 (3000 SqFt):** $3000 \times 200 = \$600,000$

3. Fill the Missing Data

House	SqFt	Price (Filled)
3	1800	**360,000**
6	3000	**600,000**

Data Analysis and Missing Data

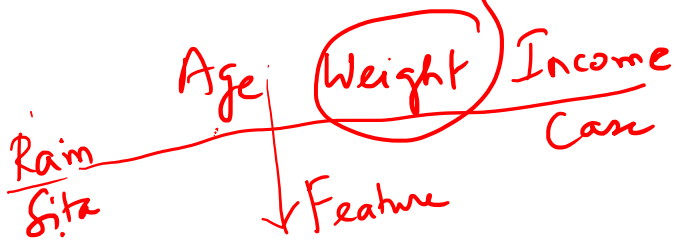
Once the pattern of missing data is determined, a decision must be made on how to obtain a complete data set for analysis. For a first step, most statisticians agree on the following guidelines.

1. If a variable is missing in a very high proportion of cases, then that variable could be deleted, but this could represent a limitation to the study that might need to be noted.

2. If a case is missing many variables that are crucial to your analysis, then that case could be deleted.

If a substantial proportion of cases have this issue, this could be a problem with the generalizability of the analysis results.

Case and Feature

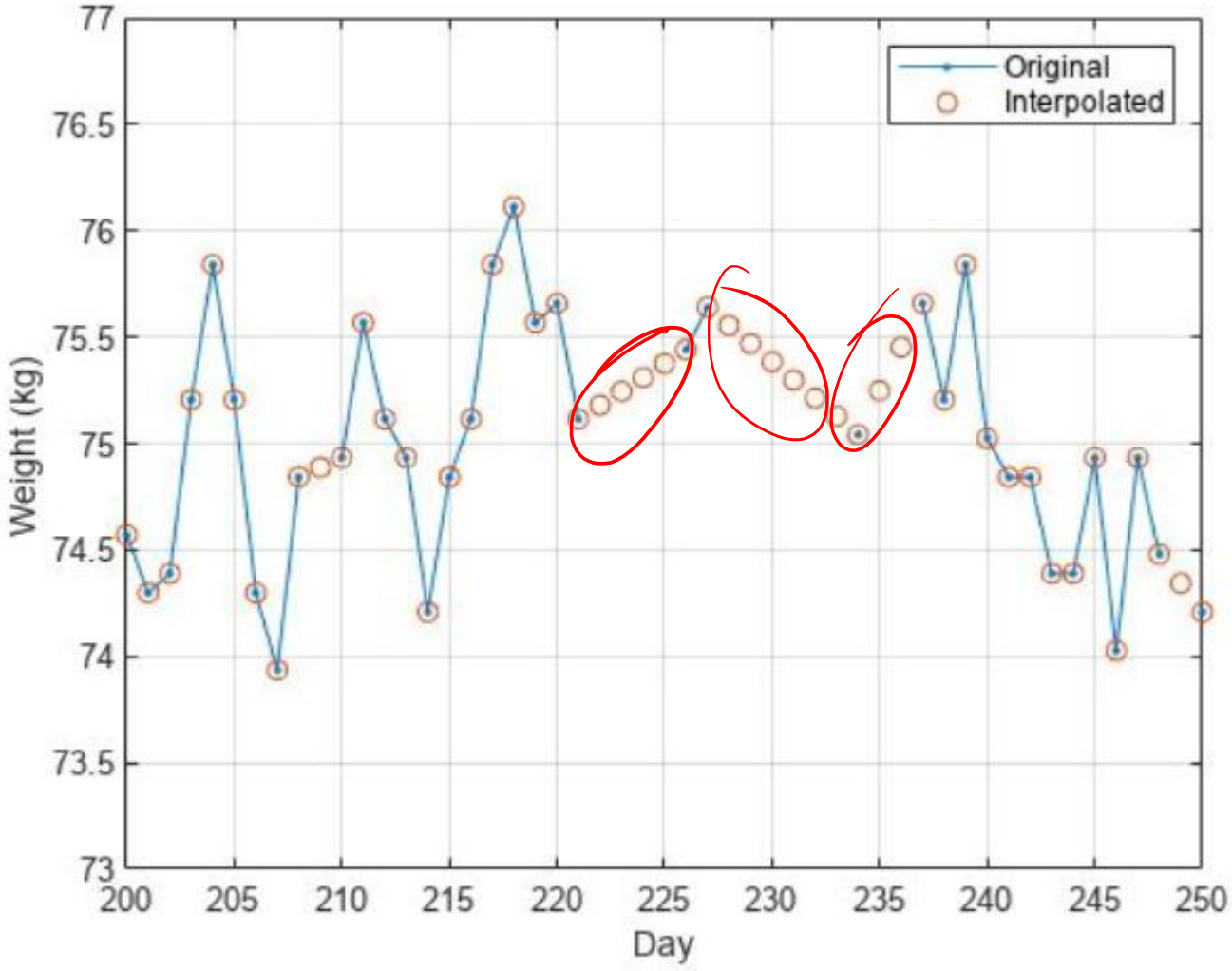


- In data analysis and machine learning, a case and a feature are exact opposites.

Manyan
Vignesh

- They represent perpendicular dimensions in a standard dataset
- Case (Observation/Row): A single entity or instance being studied. For example, in a customer dataset, a single person's profile would be one case.
- Feature (Variable/Column): An individual measurable property or characteristic of a case.
- For example, the customer's age, income, and location are all features.

Further Pointers to Data Sanitization



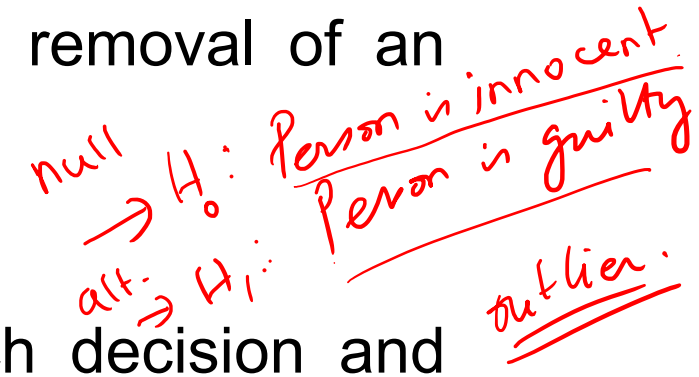
Source: Matlab Documentation R2026a, Mathworks

Detection of Outliers

- Outliers are observations that appear inconsistent with the rest of the data set
- Determining outliers by setting min. and max. values
- Using limits, extreme/unreasonable outliers prevented from entering data set
- Often, observations seem quite high or low but are not impossible.
- These values are the most difficult ones to cope with.
- Should they be removed or not?
- Statisticians differ in their opinions, from “if in doubt, throw it out” to
- Unethical to remove an outlier for fear of biasing the results
- The investigator may wish to eliminate these outliers from the analyses but report them along with the statistical analysis.

Detection of Outliers

- Run the analyses twice, both with the outliers and without them, to see if they make an appreciable difference in the results.
- Problem arises when rejecting a null hypothesis if the removal of an outlier would result in the hypothesis not being rejected.
- Whatever decision is made regarding outliers that such decision and potentially its consequences are made transparent and are justified in any report or publication.



Detection of Outliers

- Outliers can arise for different reasons. They can be due to
  - chance (random)
  - a mistake in measurement or data entry
  - an unusual event
- When faced with large no. of numerical values, tedious to find unusual ones
- Even for a small data set as the one in Table is not easy to detect possible outliers just by looking at the raw data.
- Numerical and/or graphical methods may be used for detecting outliers, with graphical methods such as histograms, boxplots, and scatterplots usually being more convenient and efficient).
- A simple numerical method for detecting outliers is the standardization of data.

```
clear all;
clc;
Data=[
1 5 0.5
2 7 -0.5
3 9 -1.5
4 11 -2.5
5 13 -3.5
6 15 -4.5
7 17 -5.5
8 19 -6.5
9 21 -7.5
10 23 -8.5
11 25 -9.5
12 27 -10.5
13 29 -11.5
14 31 -12.5
15 33 -13.5
16 35 -14.5
17 37 -15.5
18 39 -16.5
19 41 -17.5
20 43 -18.5
21 45 -19.5
22 47 -20.5
23 49 -21.5
24 51 -22.5
25 53 -23.5
26 55 -24.5
27 57 -25.5
28 59 -26.5
29 61 -27.5
];
Data_Norm=normalize(Data);
mudata =mean(Data);
stdevdata=std(Data);
Data_Norm2 = (Data - mudata)./stdevdata;
Compare =[Data_Norm Data_Norm2];
```

```
Compare =

   -1.6442   -1.6442    1.6442   -1.6442   -1.6442    1.6442
   -1.5268   -1.5268    1.5268   -1.5268   -1.5268    1.5268
   -1.4093   -1.4093    1.4093   -1.4093   -1.4093    1.4093
   -1.2919   -1.2919    1.2919   -1.2919   -1.2919    1.2919
   -1.1744   -1.1744    1.1744   -1.1744   -1.1744    1.1744
   -1.0570   -1.0570    1.0570   -1.0570   -1.0570    1.0570
   -0.9396   -0.9396    0.9396   -0.9396   -0.9396    0.9396
   -0.8221   -0.8221    0.8221   -0.8221   -0.8221    0.8221
   -0.7047   -0.7047    0.7047   -0.7047   -0.7047    0.7047
   -0.5872   -0.5872    0.5872   -0.5872   -0.5872    0.5872
   -0.4698   -0.4698    0.4698   -0.4698   -0.4698    0.4698
   -0.3523   -0.3523    0.3523   -0.3523   -0.3523    0.3523
   -0.2349   -0.2349    0.2349   -0.2349   -0.2349    0.2349
   -0.1174   -0.1174    0.1174   -0.1174   -0.1174    0.1174
         0         0         0         0         0         0
    0.1174    0.1174   -0.1174    0.1174    0.1174   -0.1174
    0.2349    0.2349   -0.2349    0.2349    0.2349   -0.2349
    0.3523    0.3523   -0.3523    0.3523    0.3523   -0.3523
    0.4698    0.4698   -0.4698    0.4698    0.4698   -0.4698
    0.5872    0.5872   -0.5872    0.5872    0.5872   -0.5872
    0.7047    0.7047   -0.7047    0.7047    0.7047   -0.7047
    0.8221    0.8221   -0.8221    0.8221    0.8221   -0.8221
    0.9396    0.9396   -0.9396    0.9396    0.9396   -0.9396
    1.0570    1.0570   -1.0570    1.0570    1.0570   -1.0570
    1.1744    1.1744   -1.1744    1.1744    1.1744   -1.1744
    1.2919    1.2919   -1.2919    1.2919    1.2919   -1.2919
    1.4093    1.4093   -1.4093    1.4093    1.4093   -1.4093
    1.5268    1.5268   -1.5268    1.5268    1.5268   -1.5268
    1.6442    1.6442   -1.6442    1.6442    1.6442   -1.6442
```

Python

```
import numpy as np
from scipy.stats import zscore

col1 = np.arange(1, 30)
col2 = np.arange(5, 63, 2)
col3 = np.arange(0.5, -28.5, -1)

Data = np.column_stack((col1, col2, col3))
#Normalization
Data_Norm = zscore(Data, ddof=1)

# Manual Normalization
mudata = np.mean(Data, axis=0)

# NumPy's std() uses N (population std dev) by default.
stdevdata = np.std(Data, axis=0, ddof=1)

Data_Norm2 = (Data - mudata) / stdevdata

# Equivalent of [Data_Norm Data_Norm2]
Compare = np.hstack((Data_Norm, Data_Norm2))

# Display the results
print("Compare =\n")
np.set_printoptions(formatter={'float': '{: 8.4f}'.format})
print(Compare)
```

```
Compare =

   -1.6442   -1.6442    1.6442   -1.6442   -1.6442    1.6442
   -1.5268   -1.5268    1.5268   -1.5268   -1.5268    1.5268
   -1.4093   -1.4093    1.4093   -1.4093   -1.4093    1.4093
   -1.2919   -1.2919    1.2919   -1.2919   -1.2919    1.2919
   -1.1744   -1.1744    1.1744   -1.1744   -1.1744    1.1744
   -1.0570   -1.0570    1.0570   -1.0570   -1.0570    1.0570
   -0.9396   -0.9396    0.9396   -0.9396   -0.9396    0.9396
   -0.8221   -0.8221    0.8221   -0.8221   -0.8221    0.8221
   -0.7047   -0.7047    0.7047   -0.7047   -0.7047    0.7047
   -0.5872   -0.5872    0.5872   -0.5872   -0.5872    0.5872
   -0.4698   -0.4698    0.4698   -0.4698   -0.4698    0.4698
   -0.3523   -0.3523    0.3523   -0.3523   -0.3523    0.3523
   -0.2349   -0.2349    0.2349   -0.2349   -0.2349    0.2349
   -0.1174   -0.1174    0.1174   -0.1174   -0.1174    0.1174
         0         0         0         0         0         0
    0.1174    0.1174   -0.1174    0.1174    0.1174   -0.1174
    0.2349    0.2349   -0.2349    0.2349    0.2349   -0.2349
    0.3523    0.3523   -0.3523    0.3523    0.3523   -0.3523
    0.4698    0.4698   -0.4698    0.4698    0.4698   -0.4698
    0.5872    0.5872   -0.5872    0.5872    0.5872   -0.5872
    0.7047    0.7047   -0.7047    0.7047    0.7047   -0.7047
    0.8221    0.8221   -0.8221    0.8221    0.8221   -0.8221
    0.9396    0.9396   -0.9396    0.9396    0.9396   -0.9396
    1.0570    1.0570   -1.0570    1.0570    1.0570   -1.0570
    1.1744    1.1744   -1.1744    1.1744    1.1744   -1.1744
    1.2919    1.2919   -1.2919    1.2919    1.2919   -1.2919
    1.4093    1.4093   -1.4093    1.4093    1.4093   -1.4093
    1.5268    1.5268   -1.5268    1.5268    1.5268   -1.5268
    1.6442    1.6442   -1.6442    1.6442    1.6442   -1.6442
```

Standardization of Data

- Table shows the observed values of two variables, X_1 and X_2, and their standardized values, called z-values $= (X_i - \mu_i)/\sigma_i$
- No normalized values exceeded 2
- Assuming the data follow a standard normal distn., a value >2 has a probability of less than 5%.
- If a normalized value is 2.56, it has a probability of less than 1%.
- Such values are unusual and we may identify it as an outlier.
- The effect of an outlier on a statistical result can be easily quantified by repeating the computations after discarding the outlier.
- Effect of discarding an outlier will be usually less impactful on the mean for large data sets
- For small sample sizes, outliers may cause substantial distortions..

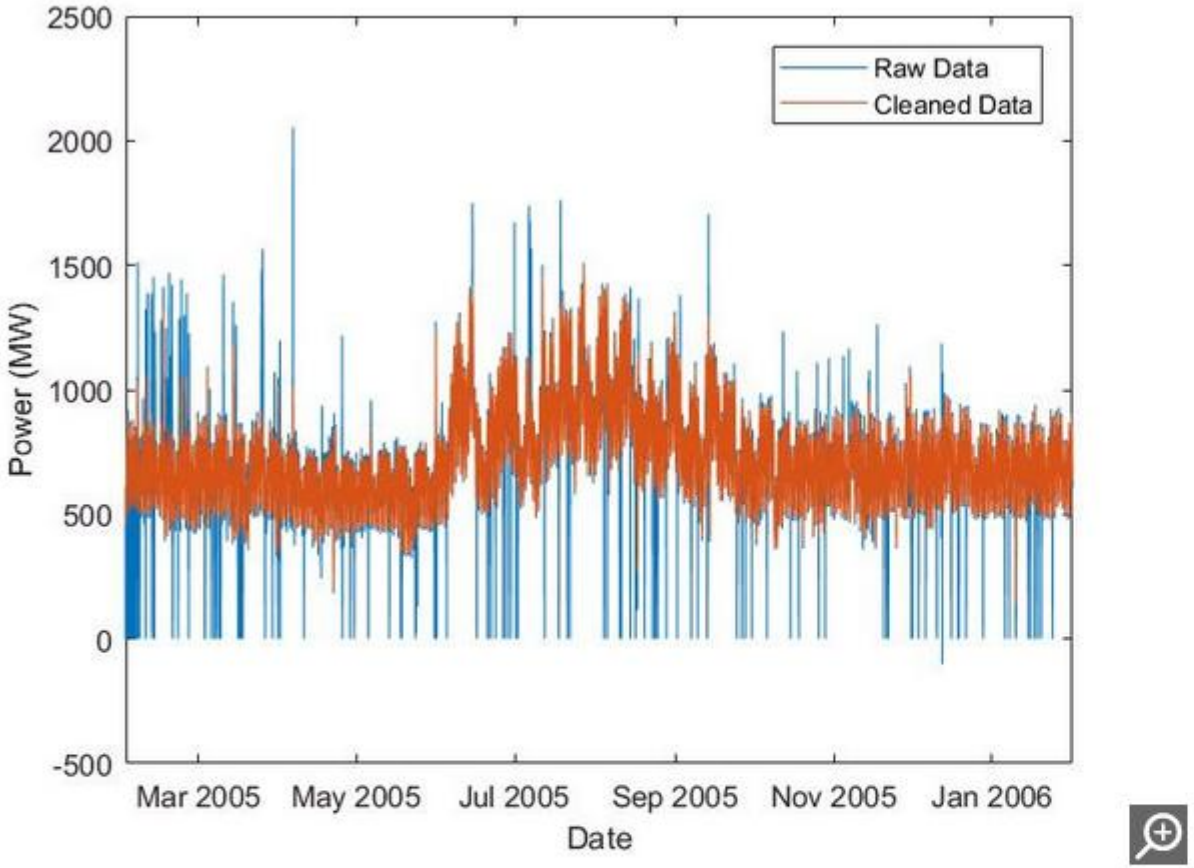


Figure 2. Plot of load consumption pre- and post-data cleaning using MATLAB.

Load consumption data cleaned using the fillmissing, filloutliers, and smooothdata functions, which is then conveyed to an AI model to produce accurate prediction of load consumption.

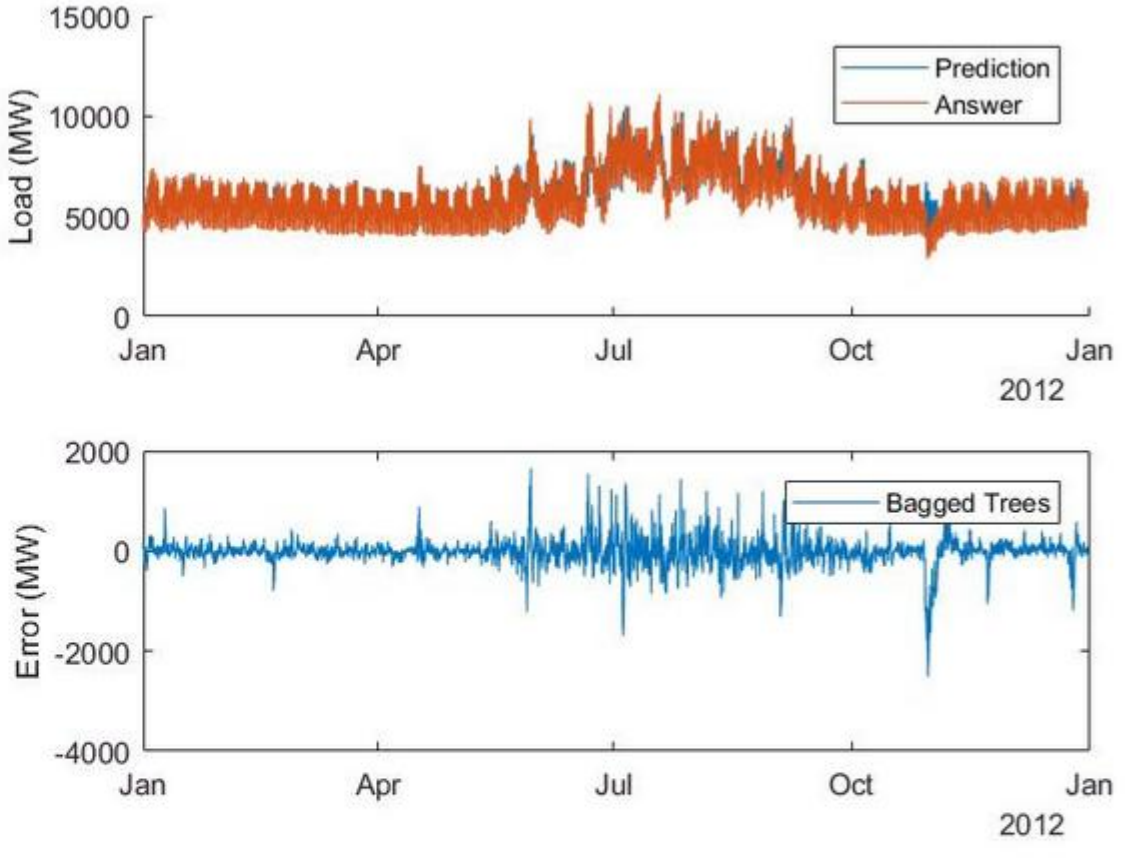
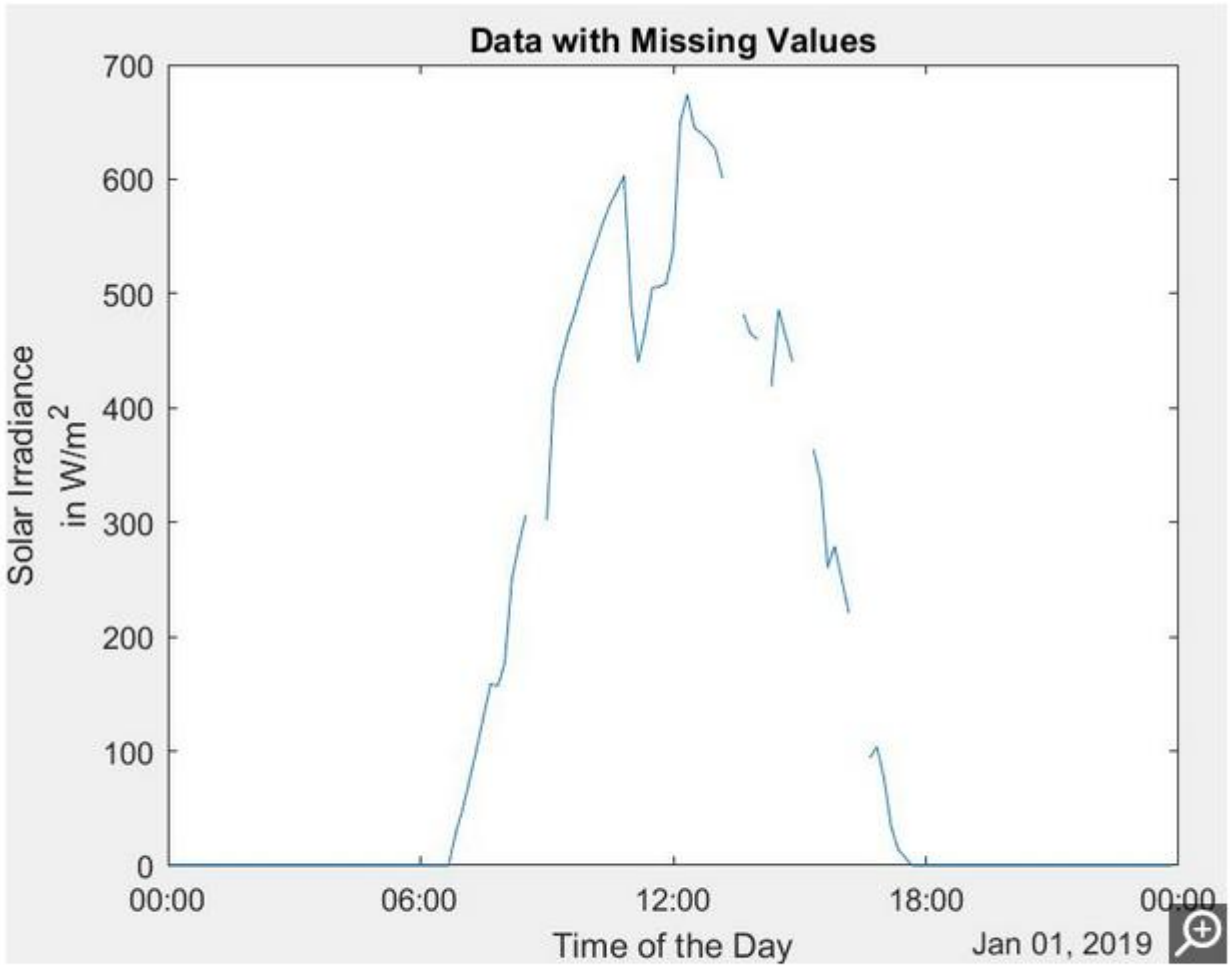


Figure 3. Validation results of an AI model that is predicting load consumption plotted using MATLAB. a) Bagged tree–based regression model with the predicted data plotted against actual consumption data. b) Error between predicted data and actual consumption data.



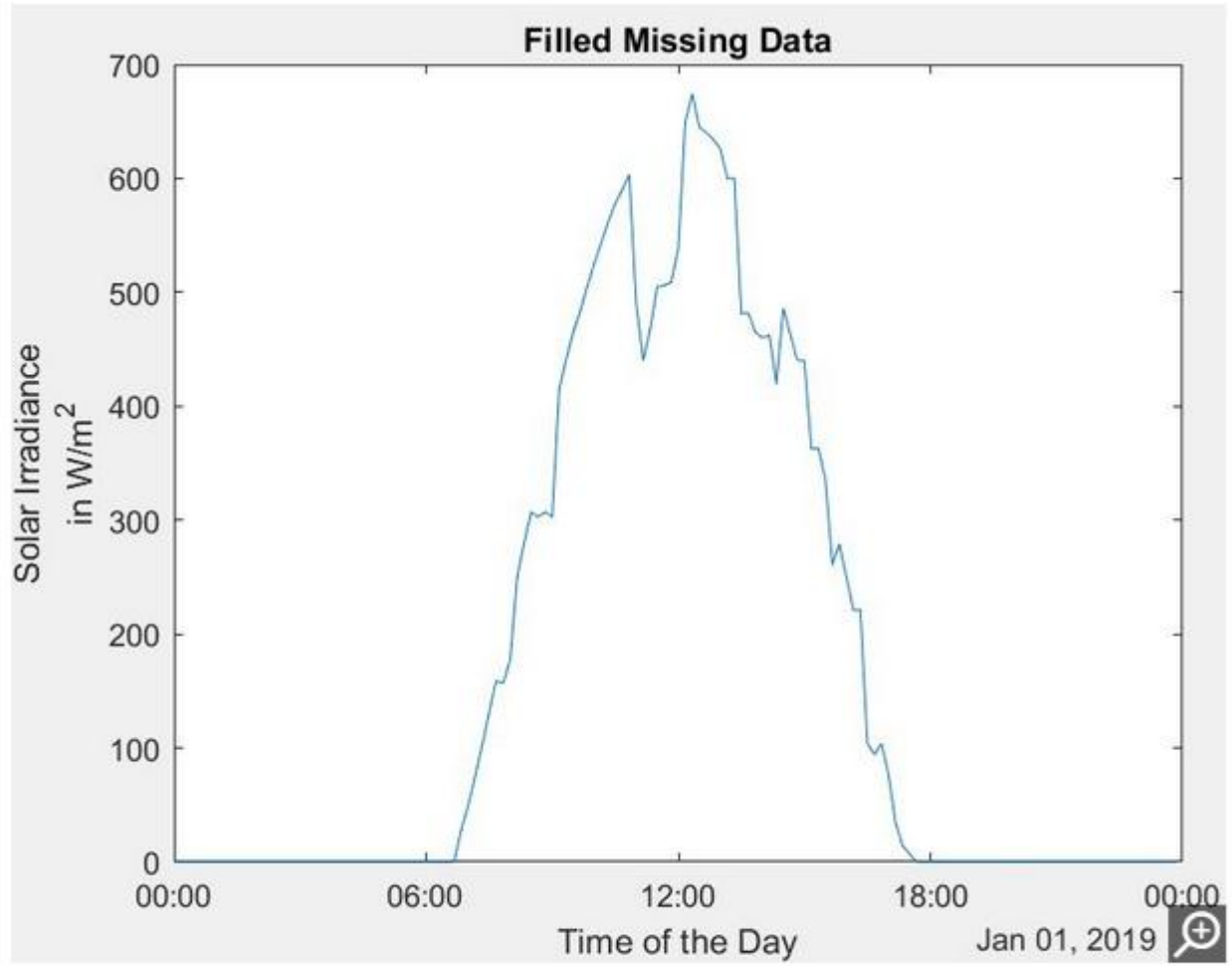
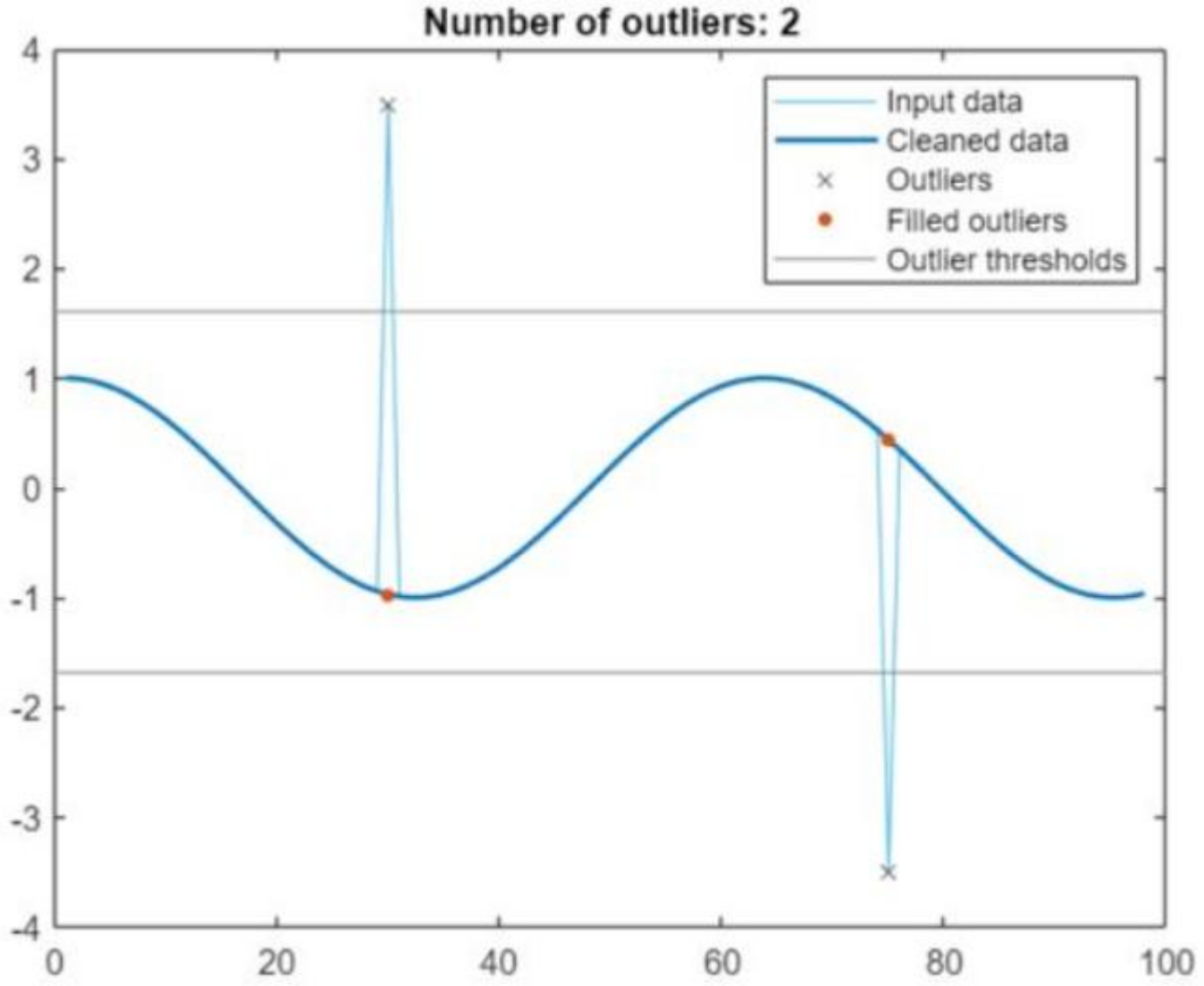


Figure 5. Time-series plot of a solar irradiance raw data set, with its missing values filled using the `fillmissing` function in MATLAB.



Clean Outlier Live Editor task used to detect and fill outliers using median thresholding and linear interpolation respectively.

Median Thresholding

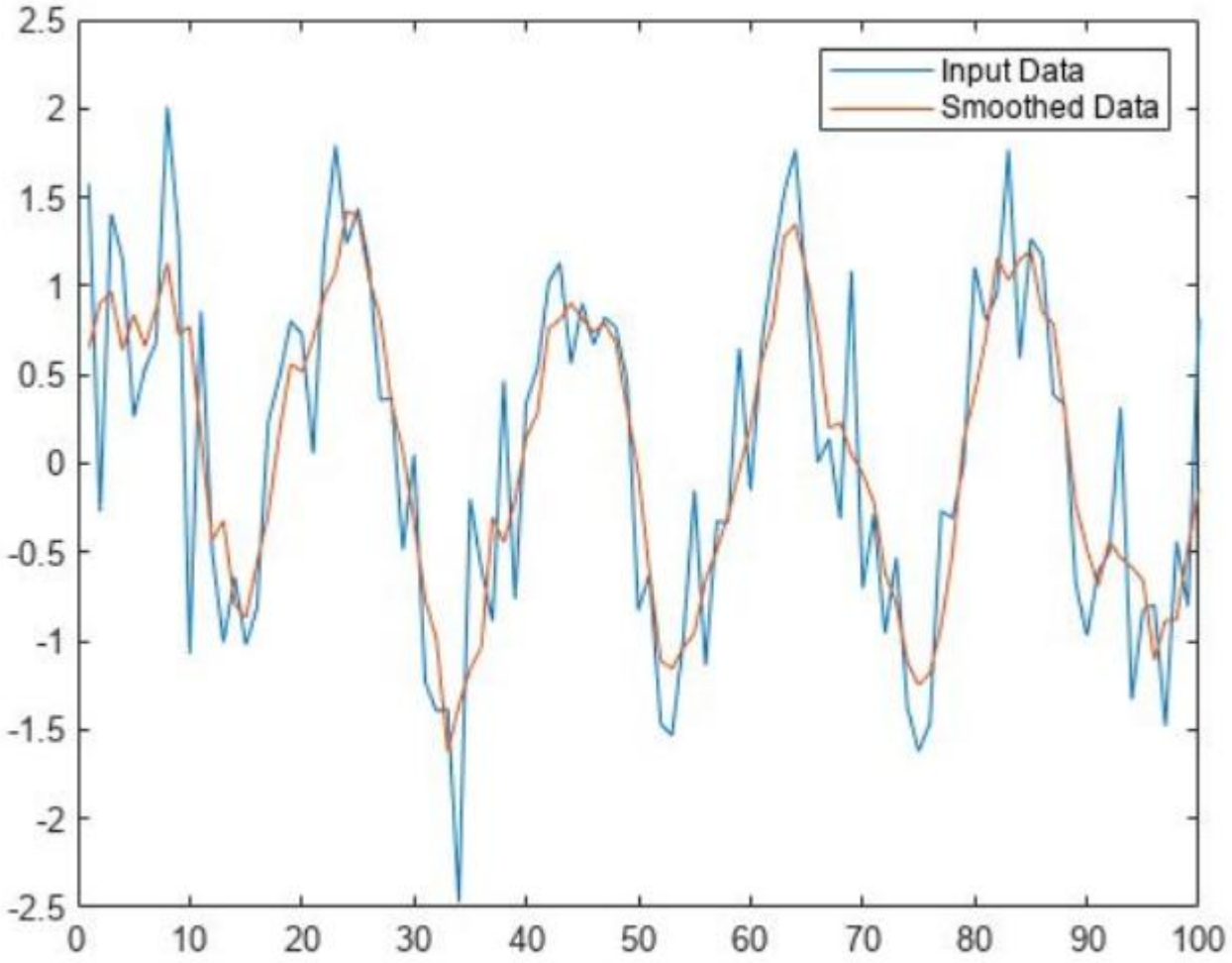
- In time series data it is a robust statistical technique primarily used for anomaly detection and noise reduction.
- It flags data points as outliers if their deviation from the rolling median exceeds a specific multiplier (often 3x) of the **Median Absolute Deviation (MAD)**.
- As it relies on the median rather than the mean, it avoids being skewed by extreme values, making it highly effective for highly volatile or noisy real-world data

How the MAD Thresholding Process Works

- **Calculate the Rolling Median:** Establish a baseline by determining the median value across a sliding window of recent time series data points
- **Calculate Absolute Deviations:** Find the absolute difference between each individual data point in the window and the calculated median.
- **Determine the MAD:** Find the median of these absolute deviations to represent the typical variability of the data
- **Apply the Threshold:** Flag any data point as an anomaly if its distance from the median is greater than a chosen threshold multiplier of the MAD. A common rule of thumb is ± 3 MADs

Ref.: https://share.google/aimode/6Laet2ovSse3ae5Ju

Data Smoothing



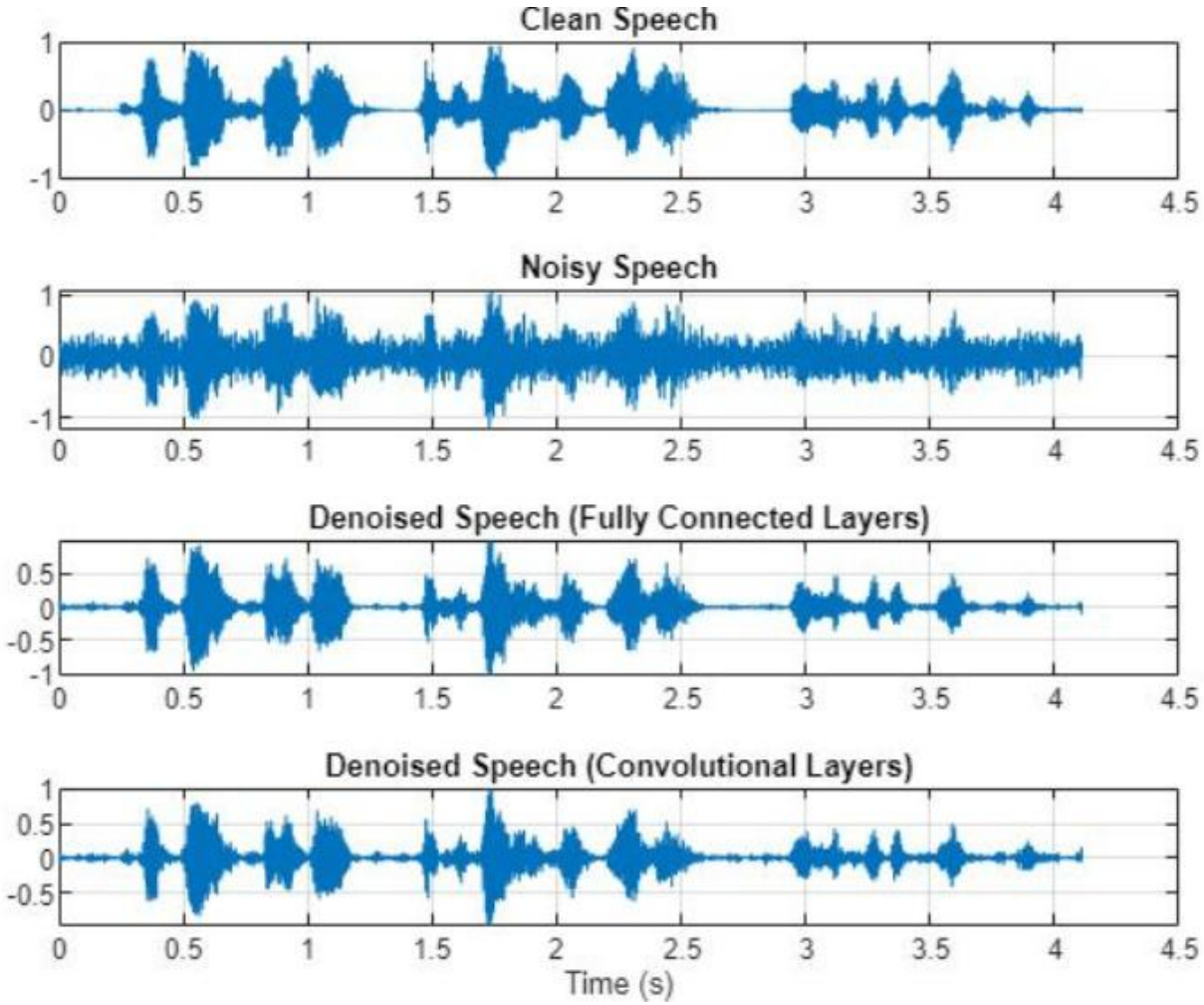
MATLAB plot of a noisy data set smoothed using a moving average filter with the smoothdata function

- Methods: Moving average filter, weighted moving average filter, or moving median-based filter Splines, Fourier transform smoothing, and Kalman filtering.
- The smoothing function requires the data set to be ordered and sampled at a fixed interval.

Data Recovery Using Deep Learning Models

- Traditional data cleaning methods work well with data that may be modeled with familiar statistical and mathematical models.
- However, for complex data sets, e.g.s human speech, EEG signals, etc., deep learning models have to be used for data cleaning.
- In this example shown in Figure (next page), speech signals with noise interference from a washing machine running in the background.
- Data cleaning methods, such as smoothing or outlier removal, may not effectively remove the noise from the washing machine data because the latter has an audio spectrum that overlaps with the speech signal.
- Deep learning networks, using fully connected or convolutional networks, are able to clean or remove noise from the speech signal, and leaving the underlying signal.

Data Recovery Using Deep Learning Models



MATLAB plots of clean and noisy speech signals and denoised output from two deep learning networks—fully connected and convolutional.